PR #221: docs(tt-cli): add a Tenstorrent CLI section

https://github.com/tenstorrent/tenstorrent.github.io/pull/221
Branch `tt-cli-docs` → `main` · author ndalyTT · head `d832e44a`

Rendered review copy of the pages this PR changes, built with Sphinx from the PR head (commit `d832e44`; a stacked PR already contains its base branch), generated 2026-10-29. Pages render in the site theme with the navigation removed; code blocks wrap instead of scrolling.

Commits (3)

`cefcc891`	2026-10-28	docs(tt-cli): add a Tenstorrent CLI reference under the Software section
`04ad9577`	2026-10-29	docs(tt-cli): move the Deploy LLMs page into the Tenstorrent CLI section
`d832e44a`	2026-10-29	docs(tt-cli): name the three serving paths behind tt serve

Files changed (19)

`core/_extra/AGENTS.md`	+3	-1
`core/_extra/llms.txt`	+7	-2
`core/_templates/home.html`	+7	-7
`core/conf.py`	+2	-0
`core/getting-started/README.md`	+1	-1
`core/getting-started/tt-software-stack.md`	+1	-1
`core/index.rst`	+1	-1
`core/software/index.rst`	+13	-3
`shared/_templates/layout.html`	+0	-1
`tt-cli/Makefile`	+56	-0
`tt-cli/conf.py`	+41	-0
`tt-cli/deploy-llms.md`	+1	-1
`tt-cli/index.md`	+105	-0
`tt-cli/launch.md`	+67	-0
`tt-cli/make.bat`	+35	-0
`tt-cli/models.md`	+116	-0
`tt-cli/serve.md`	+136	-0
`tt-cli/update.md`	+86	-0

versions.yml	+1	-0

Pages in this PDF

1. tt-cli/index.html
2. tt-cli/update.html
3. tt-cli/models.html
4. tt-cli/serve.html
5. tt-cli/launch.html
6. tt-cli/deploy-llms.html
7. index.html

Tenstorrent CLI

`tt` is the Tenstorrent command-line interface and the primary interface for operating your Tenstorrent hardware. It installs and updates the system software as one tested set, checks the devices in your machine, tells you which models they can run, serves those models as OpenAI-compatible endpoints, and connects applications to them.

Installing Tenstorrent Software walks through a first install with `tt` and keeps to the commands a new system needs. These pages go deeper into each command group for when you want to know how it works or what else it can do. If you want to develop against Tenstorrent's open-source repositories rather than operate hardware, start with TT-Metalium™, TT-NN™, or TT-Forge™.

The source is in the tt-cli repository on GitHub.

Installing `tt`

`tt` is a Python application. It needs Python 3.10 or newer with the `venv` module, plus `git` and `curl`, and it must live in its own isolated Python environment so that it can update itself safely. Tenstorrent recommends uv for that environment; follow the setup instructions at https://docs.astral.sh/uv/ to install it. Then:

```
uv tool install tenstorrent
uv tool update-shell
```

Open a new terminal or source your shell configuration file (`source ~/.bashrc` for `bash`), then confirm `tt` runs with `tt --version`. If you already use pipx, `pipx install tenstorrent` works too. Other ways to install, including from a checkout of the repository, are in the developer notes.

Ubuntu 22.04 LTS and 24.04 LTS are the supported platforms. Other Linux distributions are untested. macOS and Windows are not supported, because there is no driver for them.

Command groups

Run `tt` on its own to see the command groups. The ones you will use most:

Group	What it does
`tt update`	Installs and updates the system software stack: driver, firmware, HugePages, TT-SMI, TT-Flash, ar
`tt device`	Shows the devices in the machine with live telemetry (`status`), their firmware and board details
`tt smi`	Hands the terminal to the interactive TT-SMI interface. Press **Q** to leave it.
`tt model`	Lists the models validated for your hardware, inspects one, logs in to Hugging Face, and manages
`tt serve`	Serves a model as an OpenAI-compatible endpoint, directly or through TT-Studio.
`tt launch`	Connects a web UI or a terminal coding agent to a running endpoint.
`tt config`	Reads and writes `tt`'s own settings, such as telemetry and the update check.
`tt report`	Collects a support bundle or opens a prefilled GitHub issue.
`tt self`	Updates `tt` itself.

Every command accepts `--help`. Most also accept `--json` for machine-readable output and `--offline` to run without any network access.

`tt serve` runs TT-Inference-Server for the models in Tenstorrent's tested catalog, hands the few models only TT-Studio carries to TT-Studio, and runs community bundles published on the Hugging Face Hub through tt-model-manager. To run TT-Inference-Server directly, with its own deployment script and a vLLM endpoint you configure yourself, see Deploy LLMs.

Keeping `tt` up to date

```
tt self update
```

`tt self update` upgrades `tt` to the newest release on PyPI and prints `tt upgraded: 1.0.1 → 1.2.0` when it had something to do, or `tt 1.2.0 is the newest release.` when you are already current. `tt` also looks up its newest release about once a day and prints a notice when one is available. That lookup carries only `tt`'s own version; `tt config set update.check false`, `TT_NO_UPDATE_CHECK=1`, or `--offline` turns it off.

The system software and `tt` are updated separately: `tt update` brings the system stack and the Tenstorrent tools to the latest tested set, and `tt self update` brings `tt` itself to the latest release. The installer that `tt update` runs also installs the `tt` release that was tested with that set, which can be older than the one you have, so run `tt self update` after `tt update` if the two disagree. See Updating the stack.

Telemetry

The first time you run `tt` at a real terminal it asks once whether to share anonymous usage telemetry:

```
Enable anonymous usage telemetry? [y/n]
```

Please consider saying yes. Telemetry tells Tenstorrent which commands people use, how long they take, and which ones fail, and that is what guides what gets fixed and built next. It is strictly opt-in: nothing is collected, stored, or sent unless you say yes, and saying no means you are never asked again. The question is never asked in scripts, pipelines, CI, or under `--json`, `--quiet`, or `--offline`; non-interactive installs simply stay opted out.

If you opt in, `tt` records one event per command: the command path such as `tt device status`, the names of the options you set (never their values), an argument value only when it is on a known list the CLI holds such as catalog model names, the exit-code category and how long the command took, coarse host facts such as OS type, CPU architecture, and Python version, and a random per-install id. It never collects free-form argument values, file paths, config values, environment variable values, hostnames, usernames, error messages, stack traces, IP addresses, or location.

You can see exactly what would be sent before you decide. Set `TT_TELEMETRY_LOG_FILE` on any command and read the file, which works without opting in and uploads nothing:

```
TT_TELEMETRY_LOG_FILE=/tmp/tt-events.jsonl tt device status
```

Change your answer at any time:

```
tt config set telemetry.enabled true    # opt in
tt config set telemetry.enabled false   # opt out, and delete anything not yet uploaded
```

Setting the cross-tool `DO_NOT_TRACK` environment variable also opts out. The telemetry policy lists every property an event can carry and every switch that controls it.

Getting help

```
tt report bundle   # a redacted support bundle, attached to a prefilled email to Tenstorrent support
tt report issue    # a prefilled GitHub issue on the tt-cli repository
```

`tt report bundle` collects your environment details, a TT-SMI snapshot, and recent logs, with paths and identifiers redacted. `tt report issue` opens a GitHub issue with the environment details already filled in. For anything the documentation does not cover, raise a support request.

Updating the Stack with `tt update`

Tenstorrent's system software stack consists of several components that are regularly tested together to ensure stability: device firmware, the kernel-mode driver (TT-KMD), HugePages configuration, TT-SMI, TT-Flash, and a container runtime for serving models. `tt update` brings your system up to the latest stable, tested combination of those components in one command. On a fresh machine it installs everything. On a machine that already has Tenstorrent software it changes only what is out of date, so you can rerun it at any time to get back to a known-good state.

Preview the plan

To see what would change without changing anything, run the update with `--dry-run`:

```
tt update --dry-run
```

The plan lists each component, the version installed, the tested version, and the action `tt update` would take. The CLI labels the tested versions `golden` in its output. On a fresh machine every component is new:

```
✓ Fetching golden versions (v3.0.0)  v3.0.0
                      Update plan (goldens: golden.json v3.0.0)
```

component	kind	installed	golden	action
tt-flash	uv-tool	–	4.0.0	install
tt-inference-server	git-venv	–	v0.25.0	optional
tt-model	uv-tool	–	a7aa9cefdd3086b091b…	install
tt-smi	uv-tool	–	6.6.0	install
tt-studio	git-venv	–	v2.12.0	optional
system-stack	system	–	hugepages 1.4.1, kmd 2.11.0, sfpi 7.71.0, firmware 19.15.0 (via install.sh 3.6.2)	converge

The `kind` column says how a component is managed: `uv-tool` components are Python tools installed into their own environments, `git-venv` components are checked out from GitHub into their own virtual environments, and the `system-stack` row is everything tt-installer manages as root. Components marked `optional` are fetched automatically the first time a command needs them, so `tt update` leaves them alone unless you pass `--include-lazy`.

Run the update

```
tt update
```

`tt update` works in three phases:

1. **Checks.** It fetches the manifest of tested versions and verifies its checksum.
2. **Tools.** It installs TT-SMI, TT-Flash, and the model manager into isolated environments under `~/.local/share/tenstorrent/`, where they cannot conflict with each other or with your own Python environment.
3. **System.** It downloads the matching tt-installer release and runs it non-interactively. This phase asks for your `sudo` password and installs the base packages, the Tenstorrent package repositories, TT-KMD, HugePages, firmware, and Docker if no container runtime is present. It also installs a `tt-metalium` command that opens the TT-Metalium™ development container.

Required

Using sudo is required, so you must enter your user's password when prompted.

`tt update` also asks you to confirm before it flashes firmware, because flashing resets the devices. On a fresh machine it cannot read the current firmware yet, so the prompt is `Continue with tt update? [y/N]` with a warning that continuing may flash and reset your devices. Stop any running model workloads first, then answer **y**. On a machine that already has the driver, the prompt only appears when the firmware is older than the tested version.

On a TT-QuietBox 2 with a fast connection the whole update takes a few minutes. It ends with a summary like `Update complete: 3 tool(s) updated, 2 optional (--include-lazy), system stack converged.`

After the update

Bring `tt` itself up to date. The installer that `tt update` runs also installs the `tt` release that was tested with that set of components, which can be older than the one you had. Run `tt self update` to return to the latest release. Over time, keep both halves current: `tt update` for the system stack and tools, `tt self update` for `tt`.

Reboot after the first install. `tt update` never reboots your machine for you. If this is the first time the driver was installed on this machine, or HugePages configuration changed, reboot before continuing:

```
sudo reboot
```

If `tt update` installed Docker, it also adds your user to the `docker` group. That takes effect when you log in again, which the reboot takes care of. `tt` talks to Docker as your user, never through `sudo`; `docker run hello-world` confirms the membership is active.

Check the hardware. `tt device status` should list every Tenstorrent device in the machine with the driver version in its title. If it reports no devices, see the troubleshooting section of the install guide.

Running tt-installer yourself

`tt update` runs tt-installer for you with the tested versions. If you prefer to drive tt-installer interactively and choose each of its options, you can run it directly:

```
/bin/bash -c "$(curl -fsSL https://tenstorrent.ai/install.sh)"
```

For installing and configuring each component by hand, refer to the manual software installation guide.

Finding Models with `tt model`

`tt` knows your hardware, so it can tell you which models are validated to run on it. `tt model` lists those models, inspects one before you download anything, logs you in to Hugging Face, and manages the models that are being served.

List the models for your hardware

```
tt model list
```

The list is filtered to the device configuration `tt` detected. The rows below are an example only, trimmed from a TT-QuietBox 2 listing, which `tt` detects as a `p300x2` configuration, to show the different ways a model can appear:

```
Models (tt catalog + community) for p300x2 (detected — `tt model list --all` for every
                                device/bundle)
┌──────────────────────────┬──────────────────────┬────────┬──────────────────┬─────────┐
│ name                     │ source               │ engine │ serving profiles │ weights │
├──────────────────────────┼──────────────────────┼────────┼──────────────────┼─────────┤
│ FLUX.1-dev               │ tt-inference-server  │ media  │ p300, p300x2     │ —       │
│ Llama-3.3-70B-Instruct   │ tt-inference-server  │ vLLM   │ p150x4, p300x2   │ —       │
│ Qwen3-32B                │ tt-inference-server  │ vLLM   │ p150x4, p300x2   │ —       │
│ Qwen3.5-9B               │ tt-studio            │ vLLM   │ p150             │ —       │
│ Qwen3.8-27B              │ tt-inference-server  │ vLLM   │ p150x4, p300x2   │ —       │
│ tt-hous/qwen3.8-flash-next │ HuggingFace        │ vLLM   │ p300x2           │ —       │
│ whisper-large-v3         │ tt-inference-server  │ media  │ p150             │ —       │
└──────────────────────────┴──────────────────────┴────────┴──────────────────┴─────────┘
```

The columns tell you:

- **source**: where the model comes from. `tt-inference-server` and `tt-studio` rows are the **released catalog**, the models Tenstorrent ships and tests. `HuggingFace` rows are **community bundles** that anyone can publish to the Hugging Face Hub with the Tenstorrent model manager. Community bundles are not tested or maintained by Tenstorrent, and are served by their full `namespace/name` id. `local` marks a bundle already installed on this machine.
- **engine**: how the model is served. `vLLM` models expose an OpenAI-compatible chat API, `media` models expose image, video, or speech endpoints, and `forge` models run through the TT-Forge™ compiler.
- **serving profiles**: the board or mesh configurations the model has a serving configuration for. Your detected configuration is always among them.
- **weights**: whether the model's weights are already downloaded on this machine, and how much space they take.

Ways to narrow or widen the list:

```
tt model list --catalog                # only the released catalog
tt model list --community              # only community bundles
tt model list --include-unverified     # community bundles Tenstorrent has not reviewed, too
tt model list --type llm               # only one kind of model (llm, vlm, image, video, audio,
text_to_speech, cnn, embedding)
tt model list --hw p150x4              # the list for another configuration, without touching
the hardware
tt model list --all                    # every model on every device, not just this machine's
```

A model that is known not to work on your board is left out. A model validated on a smaller configuration than you have, such as a single-chip model on a four-chip system, is listed and runs on part of the system.

Inspect a model before you download it

```
tt model info Qwen3.8-27B
```

`tt model info` is the place to check before you serve a model. It reports the model's Hugging Face repository, the minimum disk space and host RAM it needs, whether its weights are already cached, the serving paths it supports, and its validation status on each board:

- `COMPLETE` models are fully validated.

- `FUNCTIONAL` models run and produce correct output but may not yet reach target performance.
- `EXPERIMENTAL` models are early and have not been through the full test suite.

A configuration that is missing from the list is one the model is not validated on. The `via` row lists the surfaces that can serve the model: most catalog models offer both `inference-server`, the path `tt serve` takes by default, and `studio`. A few are carried only by TT-Studio, and `tt serve` sends those to it automatically.

Log in to Hugging Face

Catalog models download their weights from the Hugging Face Hub, so serving needs a Hugging Face account and login, even for freely available models. Some models are also gated: you must visit the model's page on the Hub and accept its license before you can download it, and the owner may take days to grant access. Meta's Llama, Google's Gemma, and Black Forest Labs' FLUX.1 models are gated; Qwen, Wan, and Whisper models are not. The `hf_repo` line in `tt model info` names the page to visit.

```
tt model login
```

`tt model login` offers two ways to log in. **Log in with your browser** shows a URL and a short code: open the URL on any device, enter the code, and approve the request. **Paste an access token** takes a read token you create at huggingface.co/settings/tokens. Either way, the login is saved in Hugging Face's own token store, so it is also picked up by any other Hugging Face tooling you use, and `tt serve --dry-run` reports it as `HF token: from the Hugging Face login store`.

Download ahead of time

```
tt model pull Qwen3.8-27B
```

`tt model pull` downloads a model's weights without serving it, so the first serve is faster. It is also a quick way to confirm that your access to a gated model works. Weights are cached under `~/.cache/huggingface` and shared by every path that serves the model, including TT-Studio.

Community bundles

Community bundles are models that anyone has packaged for Tenstorrent hardware and published on the Hugging Face Hub. To find bundles for a specific model, search the Hub:

```
tt model search gpt-oss
```

Bundles are served through the same commands as catalog models, with the bundle's repository id in place of a catalog name:

```
tt model info <namespace>/<name>    # the bundle's manifest and a compatibility verdict
tt serve <namespace>/<name>         # served through the Tenstorrent model manager
```

`tt` checks that the bundle was built for your chip architecture and that your system has enough chips before it serves, and asks for confirmation before it serves a bundle that has not been verified. Tenstorrent does not test or maintain community bundles. The community models guide explains how bundles are packaged, published, and pulled, and how to publish one of your own.

Manage what is running and what is on disk

```
tt model ps                     # what is being served, on which port, and whether it is healthy
tt model logs <model>           # the server's log (--follow to stream it)
tt model stop <model>           # stop the server; the weights stay cached
tt model rm <model>             # remove a model's local artifacts, keeping its weights
tt model rm <model> --include-weights   # remove the weights too
```

`tt model curl` sends a chat completion to a running LLM server; see Serving models.

Serving Models with `tt serve`

Serving a model gives you an OpenAI-compatible API endpoint running on your Tenstorrent hardware. `tt serve <model>` does it in one command for any model in `tt model list`, whether it is a large language model, a diffusion model, a speech model, or a vision model, directly or through TT-Studio's web interface.

Preview with `--dry-run`

```
tt serve Qwen3.8-27B --dry-run
```

`tt serve <model> --dry-run` shows how the model would be served without starting anything: the backend, the device configuration `tt` detected, the container image, where the weights go, which Hugging Face login it found, and the port. Below the table it prints the exact command it would run. It is a quick way to confirm `tt` detected your hardware the way you expect.

Serve

```
tt serve Qwen3.8-27B
```

`tt serve` picks the serving path from the model name: TT-Inference-Server for catalog models, TT-Studio for the few only it carries, and tt-model-manager for a community bundle id. For a catalog model it installs TT-Inference-Server on first use, pulls the model's container image, downloads the weights, opens the devices, loads the weights, and warms the model up. It shows those stages as a live checklist with their elapsed times and returns only once the endpoint actually answers requests. A first serve of Qwen3.8-27B on a TT-QuietBox 2 looks like this:

```
  ✓ host ready  tt-inference-server 0.25.0
  ✓ weights ready  Qwen/Qwen3.8-27B  6m 53s
  ✓ image ready  vllm-tt-metal-src-release-ubuntu-22.04-amd64:0.25.0  1m 37s
  ✓ container started  tt-inference-server-d02c18d2  3.8s
  ✓ engine started  vLLM 0.26.0  17.8s
  ✓ Tenstorrent device opened  4 chips · mesh (1, 4)  7.0s
  ✓ KV cache configured  1,051,104 tokens  3m 28s
  ✓ model warmed up  209s of warmup
  ✓ API server started  1.8s
  ✓ endpoint answering
✓ Phase 3/3 · Start  15m 26s
┌─ Qwen3.8-27B ready ──────────────────────────────────────────────────────────┐
│                                                                              │
│  endpoint  http://127.0.0.1:20000/v1                                         │
│  models    curl http://127.0.0.1:20000/v1/models                             │
│  chat      tt launch                                                         │
│  Ready in 15m 26s · via tt-inference-server                                  │
│  Logs · tt model logs Qwen3.8-27B --follow                                   │
│  Stop · tt model stop Qwen3.8-27B                                            │
└──────────────────────────────────────────────────────────────────────────────┘
```

The stages are the same for every model of a kind; the times are not. Most of a first serve is the download: here the container image (about 18 GB) and the weights (56 GB) took nine minutes, then the model was compiled and warmed up on the hardware. Everything downloaded and compiled is cached, so later serves of the same model are faster: about seven minutes for Qwen3.8-27B on a TT-QuietBox 2, most of it sizing the KV cache and warming up. `tt model pull <model>` downloads the weights ahead of time. A media model's checklist has fewer stages: it loads the model and starts answering, without a KV cache or warmup.

To watch the raw server log while you wait, run `tt model logs <model> --follow` in a second terminal. Pressing Ctrl-C in the `tt serve` terminal stops watching, not the server. `tt serve` waits up to an hour for the endpoint to answer, then stops watching and leaves the server running; set `TT_SERVE_READY_TIMEOUT` to a number of seconds to wait longer.

Send a first request

The endpoint speaks the OpenAI API on port 20000, so any OpenAI-compatible client can use it: point the client at `http://localhost:20000/v1` with any value for the API key. For an LLM, `tt model curl` sends a chat completion without any JSON on your part:

```
tt model curl "In one sentence, what is Tenstorrent?" --max-tokens 2048
```

`tt model curl` asks the server which model it is serving, sends your prompt, and prints the server's JSON chat completion. Any option it does not recognize goes straight into the request body, which is how `--max-tokens` reaches the server. Qwen3.8-27B is a reasoning model: it thinks first, in a separate `reasoning` field of the reply, and then answers in `content`, and its default budget of 64 tokens is used up by the thinking, which is why the example raises it. Add `--print` to see the equivalent `curl` command instead of sending it.

Which route an application calls depends on the kind of model:

Kind of model	Route
Large language and vision-language models	`/v1/chat/completions`
Speech recognition	`/v1/audio/transcriptions`
Text to speech	`/v1/audio/speech`
Image generation	`/v1/images/generations`
Video generation	`/v1/videos/generations`, polled at `/v1/videos/generations/<j`
Embeddings	`/v1/embeddings`

Options

```
tt serve <model> --port 20001        # serve on a port other than 20000
tt serve <model> --device p150x4     # override the detected device configuration
tt serve <model> -- <server args>    # anything after -- is passed through to the server itself
tt serve <model> --studio            # serve through TT-Studio instead of directly
```

Manage what is running

```
tt model ps              # what is being served, on which port, and whether it is healthy
tt model logs <model>    # the server's log (--follow to stream it)
tt model stop <model>    # stop the server; the weights stay cached
```

A model server that was stopped or crashed can leave the devices in a state the next server cannot use. The server log then ends with a `TT_THROW` message that asks you to reset the board. `tt device reset` resets every device; it asks for confirmation because it interrupts any running workload, and

`tt device status` should show the devices back at their idle clock afterwards.

Serve through TT-Studio

TT-Studio is a web application that serves the same catalog and gives you a browser interface for each kind of model: chat with knowledge bases for language models, image and video generation for diffusion models, speech-to-text for recognition models, object detection for vision models, and a voice agent that chains them. Every model TT-Studio serves is also on an API endpoint on port 20000, so applications can use it at the same time as the browser does. TT-Studio requires Docker.

```
tt serve Qwen3.8-27B --studio
```

TT-Studio takes port 3000 and the devices for itself, so stop anything `tt serve` or `tt launch` started first. A model server that was stopped by `tt model stop` leaves the devices in a state TT-Studio cannot initialize, so reset them before you start:

```
tt launch stop openwebui
tt model stop Qwen3.8-27B
tt device reset
tt serve Qwen3.8-27B --studio
```

On first use `tt` fetches the latest TT-Studio release. TT-Studio asks you to accept its terms and offers to add a `tt-studio` shortcut to your shell. It then brings up its services and prints a `TT Studio is ready` box with its web UI at `http://localhost:3000`, serves the model, and prints a ready box with the model's endpoint and the **Models Deployed** page at `http://localhost:3000/models-deployed`, which lists every running model with its health, the chips it occupies, its log, and a control to delete it. With the images already downloaded this takes a few minutes. Without a model name, `tt serve --studio` lists everything TT-Studio can run on your system and asks which one you want.

`tt model stop <model>` stops a model that TT-Studio is serving, resets the chips it was using, and takes TT-Studio's services down with it, keeping its settings, downloaded weights, chat history, and knowledge bases for next time. With the `tt-studio` shortcut you can also manage TT-Studio directly:

```
tt-studio --stop-model <model>   # stop one model and keep TT-Studio running
tt-studio --status               # a live view of the running stack
tt-studio --logs                 # stream the logs of every TT-Studio service
tt-studio --stop                 # stop TT-Studio, keeping its data
```

Troubleshooting

`tt serve` **stops before starting, asking for a Hugging Face token.** Catalog models require a Hugging Face login to download their weights. Run `tt model login`; see Finding models.

`tt serve` **fails with "Access to model … is restricted", or the download stops with "requires approval".** The model is gated and your Hugging Face account has not been granted access yet, or the login on this machine is for a different account. Accept the license on the model's Hub page, wait for the owner to approve the request if the model requires that, check `tt model login`, and serve again. `tt model logs <model>` shows the server's own output for any failed start.

A serve fails with "Try resetting the board". Run `tt device reset` and serve again, as described above.

`tt serve` **says the model did not report ready within 60 minutes.** A cold start of a large model compiles for your hardware and can be slow. Check whether it is still making progress with `tt model logs <model> --follow`; `tt model ps` reports it `healthy` once it answers. If the log shows an error, stop the server with `tt model stop <model>`.

TT-Studio does not start because port 3000 is in use. TT-Studio needs port 3000 and does not move to another one. Stop whatever is using it first, for example `tt launch stop openwebui`.

Running out of disk space. `tt model rm <model>` removes a model's local artifacts and keeps its weights; add `--include-weights` to remove those too. The weights live under `~/.cache/huggingface`.

Connecting Apps with `tt launch`

An API endpoint is the foundation. `tt launch` connects a ready-made application to a model that `tt serve` is serving, so you can start using the model right away without writing any configuration yourself.

See which clients are available

```
tt launch list
```

```
                                        tt launch
┌─────────────┬──────────────┬──────────────────┬──────────────────┬──────────────────────────────────────────
──────┐
│ client      │ kind         │ model            │ status           │ configures
│
├─────────────┼──────────────┼──────────────────┼──────────────────┼──────────────────────────────────────────
──────┤
│ opencode    │ terminal     │ tool calling     │ needs opencode   │
~/.config/opencode/opencode.json       │
│ pi          │ terminal     │ tool calling     │ needs pi         │ ~/.pi/agent/models.json
│
│ aider       │ terminal     │ tool calling     │ needs aider      │ environment (this run only)
│
│ qwencode    │ terminal     │ tool calling     │ needs qwen       │ command-line arguments (this
run only) │
│ hermes      │ terminal     │ tool calling     │ needs hermes     │ environment (this run only)
│
│ openwebui   │ web service  │ any chat model   │ ready            │ container tt-open-webui
│
│ anythingllm │ web service  │ any chat model   │ ready            │ container tt-anythingllm
│
└─────────────┴──────────────┴──────────────────┴──────────────────┴──────────────────────────────────────────
──────┘
```

There are two kinds of client:

- **Web UIs** such as Open WebUI and AnythingLLM. `tt` pulls and runs these for you in a container, after asking once. They show as `ready` as soon as Docker is available, and work with any chat model.
- **Terminal coding agents** such as OpenCode, Aider, pi, Qwen Code, and Hermes Agent. You install the agent yourself, and `tt` writes the small configuration it needs to reach your model, in the location the `configures` column names. These need a model that supports tool calling, which `tt` checks for you. A client whose `status` is `needs <command>` is one you have not installed yet.

Launch a web UI

With a model being served, launch a chat interface in your browser:

```
tt launch openwebui
```

`tt` asks the server which model it is serving, then asks you to confirm before it pulls the image: `Pull ghcr.io/open-webui/open-webui:main (a few GB) and run it as tt-open-webui on port 3000. Continue? [y/N]`. It starts the container, prints a summary of what it connected, and ends with `tt-open-webui is ready at http://localhost:3000`. Open that URL in your browser. No sign-in is needed: pick the model and start chatting with it running on your Tenstorrent hardware.

```
tt launch stop openwebui   # stop the web UI; its chat history is kept in a volume
```

If another service already holds port 3000, for example TT-Studio or a development server of your own, `tt launch` says so. The web UI container keeps the port it was created with, so `tt launch` prints the exact commands to recreate it on another port: `docker rm tt-open-webui && tt launch openwebui --web-port 3001`. The container's chat history lives in a separate volume and survives the recreate.

Launch a terminal coding agent

```
tt launch opencode
```

For a terminal client, `tt` confirms the served model supports tool calling, writes the client's configuration to point at the endpoint, and starts the client. For `aider`, `qwencode`, and `hermes` the configuration is passed through the environment or the command line and applies to that run only; for `opencode` and `pi` it is written to the client's configuration file.

The full list of clients, what each one needs installed, and the options for connecting to a server on another machine are in the client launchers guide.

Prefer a point-and-click experience?

TT-Studio serves models and gives you a browser interface for them in one place, without a separate `tt launch` step. It is a serving path rather than a client, so you start it with `tt serve <model> --studio`; see Serving models.

Deploy LLMs

This page demonstrates how to deploy LLMs using the tt-inference-server project. We currently use vLLM to serve LLMs for production applications. It is also a convenient entry-point into Tenstorrent's software ecosystem. You will learn how to prepare your Tenstorrent system, configure access to gated models on Hugging Face, and deploy a vLLM-powered API endpoint using the tt-inference-server project.

Before You Begin

Before beginning this procedure, ensure that you have completed the base software installation. This process has specific system and hardware requirements.

Important

- This guide assumes that you have already followed Installing Tenstorrent Software.
- Deploying the recommended models requires a minimum of 360 GB of free disk space in your root partition.

(Wormhole™ only)

If you are using a TT-QuietBox® or TT-LoudBox™ you must complete the following step

For these systems you must configure a system-level mesh topology between the Wormhole™ Networked AI Processors. Run the following script to install `tt-topology` and configure the mesh.

```
TMP_DIR=$(mktemp -d); (trap 'echo "---"; echo "Cleaning up..."; if type deactivate &>/dev/null; then deactivate; fi; echo "Removing temporary directory: $TMP_DIR"; rm -rf "$TMP_DIR"; cd; echo "Cleanup complete."' EXIT; trap 'echo -e "\033[0;31m!!! ERROR: Failed to configure mesh topology\033[0m"' ERR; set -e; cd "$TMP_DIR"; echo "Working in temporary directory: $TMP_DIR"; echo "---"; echo "Creating Python virtual environment..."; python3 -m venv tt-topology-venv; source tt-topology-venv/bin/activate; echo "Virtual environment activated."; echo "---"; echo "Installing tt-topology from git..."; pip install --quiet git+https://github.com/tenstorrent/tt-topology.git; echo "tt-topology installed."; echo "---"; echo "Running tt-topology command. This may take a moment..."; tt-topology -l mesh; echo "---"; echo "Script finished successfully.";)
```

Installing Docker

tt-inference-server requires Docker to be installed. To install Docker, please follow the official installation instructions.

Verify that the installation was successful by running the `hello-world` image:

```
sudo docker run hello-world
```

You must also follow the official post-installation instructions to run Docker without root permissions.

Verify that the post-installation was successful by running the `hello-world` image again, this time without root permissions:

```
docker run hello-world
```

Important

Do not continue if you cannot run the `hello-world` image without root permissions, be sure to follow all post-installation instructions.

Step 1: Getting Model Access on Hugging Face

The recommended large language models are gated and require a Hugging Face account.

1. Request Access to the Model

Visit the model's page on Hugging Face and follow the instructions to request access.

- For TT-QuietBox® and TT-LoudBox™ systems, we recommend meta-llama/Llama-3.3-70B-Instruct
- For add-in-card products (n-series, p-series), we recommend meta-llama/Llama-3.1-8B-Instruct

For a full list of the currently available and tested models, please visit the tt-inference-server GitHub page.

Important

Access is granted by the model owner and is not controlled by Tenstorrent. This process may take several days.

2. Create a Hugging Face Access Token

Once you have access, generate an access token with a minimum of **read** permissions. This token is required to download the model's weights from Hugging Face.

3. Export the Token

On the system where you will deploy the server, export your token as an environment variable.

```
export HF_TOKEN="<your-hugging-face-access-token>"
```

Step 2: Configuring the vLLM Server

1. Select Your Hardware

Run the following script to specify your hardware. This script sets the required environment variables and selects the recommended model for your system.

```
select_device_and_model(){ echo -e "\nSelect a Tenstorrent system from the list below:"; PS3=$'\n#? '; options=("TT-QuietBox (Wormhole)" "TT-QuietBox (Blackhole)" "TT-LoudBox (Wormhole)" "TT-LoudBox (Blackhole)" "n150s" "n150d" "n300s" "n300d" "p100a" "p150a" "p150b" "Quit"); select opt in "${options[@]}"; do case "$opt" in "TT-QuietBox (Wormhole)") DEVICE="t3k"; MODEL="Llama-3.3-70B-Instruct";; "TT-QuietBox (Blackhole)") DEVICE="p150x4"; MODEL="Llama-3.3-70B-Instruct";; "TT-LoudBox (Wormhole)") DEVICE="t3k"; MODEL="Llama-3.3-70B-Instruct";; "TT-LoudBox (Blackhole)") DEVICE="p150x8"; MODEL="Llama-3.3-70B-Instruct";; "n150s"|"n150d") DEVICE="n150"; MODEL="Llama-3.1-8B-Instruct";; "n300s"|"n300d") DEVICE="n300"; MODEL="Llama-3.1-8B-Instruct";; "p100a") DEVICE="p100"; MODEL="Llama-3.1-8B-Instruct";; "p150a"|"p150b") DEVICE="p150"; MODEL="Llama-3.1-8B-Instruct";; "Quit") echo "❌ Exiting without setting any environment variables."; return;; *) echo "❌ Invalid option. Try again."; continue; esac; export DEVICE MODEL; echo -e "\n✅ DEVICE set to '$DEVICE'"; echo "✅ MODEL set to '$MODEL'"; break; done; }; select_device_and_model
```

2. Check Model Access

Execute this script to confirm you can access the recommended model's weights:

```
check_hf_access() { [ -z "$MODEL" ] && { printf "✖ Error: Please provide a Hugging Face repository ID.\n"; return 1; }; ! command -v curl &>/dev/null && { printf "✖ Error: curl is not installed.\n"; return 1; }; local REPO_ID="meta-llama/$MODEL"; local TOKEN=${HF_TOKEN:-$(cat "$HOME/.cache/huggingface/token" 2>/dev/null)}; [ -z "$TOKEN" ] && printf "ℹ️ Info: No Hugging Face token found.\n   You can only access public repositories.\n"; local AUTH_HEADER=""; [ -n "$TOKEN" ] && AUTH_HEADER="Authorization: Bearer $TOKEN"; printf "Checking access for: %s...\n" "$REPO_ID"; local URL="https://huggingface.co/$REPO_ID/resolve/main/config.json"; local HTTP_CODE=$(curl -s -L -o /dev/null -w "%{http_code}" -H "$AUTH_HEADER" "$URL"); case $HTTP_CODE in 200) printf "✔ Access granted.\n";; 401) printf "✖ Access denied (401 Unauthorized).\n   This is a private or gated repository.\n   Ensure your token is valid and has the correct permissions.\n";; 403) printf "✖ Access forbidden (403 Forbidden).\n   The repository is gated.\n   You need to visit the repository page on Hugging Face and request access.\n";; 404) printf "✖ Repository or 'config.json' not found (404 Not Found).\n   Please check if the repository ID '$REPO_ID' is correct.\n";; *) printf "✖ Failed to check access.\n   Received HTTP status code: %s\n" "$HTTP_CODE";; esac; }; HF_HUB_DISABLE_XET=1; check_hf_access;
```

If the command does not succeed and print `✔ Access granted.`, please make sure you have exported your Hugging Face token as per the above instructions.

3. Clone the Server Repository

```
# 1. Clone the repository
git clone https://github.com/tenstorrent/tt-inference-server.git
cd tt-inference-server

# 2. Update all tags from the remote (standardizing the local list)
git fetch --tags

# 3. Checkout the highest Semantic Version tag starting with "v"
git checkout $(git tag -l "v*" --sort=-v:refname | head -n 1)
```

Step 3: Launching the vLLM Server

1. Set the JWT Secret

This string is used to seed the generation of your server's API key.

```
export JWT_SECRET="testing"
```

2. Run the Deployment Script

Execute the following command. The script prompts you for configuration details; in most cases, you may accept the default values.

```
python3 run.py --model $MODEL --device $DEVICE --workflow server --docker-server
```

Important

The first time you run this command, it will download the model's weights. This download can take more than 30 minutes.

3. Wait for the server to initialize

After the download completes, the server will start initializing in a docker container.

Important

The first time you start the server, the initialization process for a 70B model should take about 40 minutes. For an 8B model it should take about 10 minutes.

Step 4: Testing the Server Endpoint

1. Check Server Health

Use the following command to check if the server is ready.

```
check_server_health(){ code=$(curl -s -o /dev/null -w "%{http_code}"
http://localhost:8000/health); exit_code=$?; if [[ $exit_code -ne 0 ]]; then echo "❌ Error:
Unable to connect to server at localhost:8000"; elif [[ $code -eq 200 ]]; then echo "✅ Server
is ready (HTTP 200)"; else echo "⚠️ Server responded with status: $code"; fi; };
check_server_health
```

Wait until the output indicates `✅ Server is ready (HTTP 200)`.

2. Generate an API Key

Your `JWT_SECRET` is used to create an API key for authenticating requests.

```
python3 -m venv request-venv
source request-venv/bin/activate
pip3 install --upgrade pip
pip install pyjwt==2.7.0
export VLLM_API_KEY=$(python3 -c 'import os; import json; import jwt; json_payload =
json.loads("{\"team_id\": \"tenstorrent\", \"token_id\": \"debug-test\"}"); encoded_jwt =
jwt.encode(json_payload, os.environ["JWT_SECRET"], algorithm="HS256"); print(encoded_jwt)')
```

3. Send an Example Request

The vLLM server exposes an OpenAI-compatible API. The first request will be slow as it performs a final warmup.

```
# Warmup request
curl -sS "http://localhost:8000/v1/completions" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $VLLM_API_KEY" \
  -d "{
    \"model\": \"meta-llama/$MODEL\",
    \"prompt\": \"San Francisco is a\",
    \"max_tokens\": 50,
    \"temperature\": 0
  }" | jq
```

Run the command again to observe the server respond at full speed.

What to Do Next

Now that your model endpoint is running:

- **Watch it in real time** — run `tt-toplike` in a second terminal window. TT-Toplike shows live core activity, power draw, and memory traffic on the chips while they serve requests.
- **Add a browser UI** — TT-Studio wraps TT-Inference-Server with a point-and-click deployment and chat interface. Run `tt-studio` in the terminal to start it.
- **Check supported models** — the tt-inference-server repository is the authoritative source for which models are validated on each hardware generation.
- **Use it as a coding assistant** — the Coding Assistant with Aider lesson walks through pairing a local vLLM endpoint with Aider for on-device pair programming.

- **Build agents on top of it** — if you have a TT-QuietBox® 2, the Local AI Agents on QuietBox 2 lesson covers tool-calling pipelines and multi-agent workflows against your local endpoint.
- **Try image and video generation** — if you have a TT-QuietBox® 2, TT-Local-Generator runs alongside inference with its own queue and gallery.

Need Additional Support?

If you encounter any issues, or have a question that isn't covered in the documentation, please raise a support request. Our team will review your request and provide assistance.

Tenstorrent Documentation

No hardware yet? Get access to Tenstorrent silicon right in your browser with Tenstorrent Cloud Console

GET STARTED

Installing Tenstorrent Software

Set up drivers, firmware, and the TT-Metalium environment on your Tenstorrent hardware, then verify your install.

TT-VSCODE-TOOLKIT

Learn by Building

Get hands-on with a guided learning path — from making your first inference to building full applications with LLMs.

Popular Destinations

TT-Studio

Point-and-click web UI for deploying and interacting with models.

TT-Inference-Server

Deploy and serve models — the canonical source for validated model support.

TT-Metalium

Low-level C++ SDK for writing custom kernels on Tensix cores.

Get Started →

Overview

How TT-Forge, TT-NN, and TT-Metalium fit together — and which to use when.

Installing Tenstorrent Software

Install drivers, firmware, and the TT-Metalium environment, then verify your setup.

Manual Installation

Manually install system dependencies, drivers, and the software stack step by step.

Learn by Building

Get hands-on with guided, end-to-end lessons in the TT-VSCode-Toolkit.

Software →

Tenstorrent CLI

The tt command line: install and update the stack, check your devices, serve models, and connect apps to them.

TT-Forge™

End-to-end MLIR compiler. Compile models from PyTorch, JAX, and ONNX.

TT-XLA

PJRT-based frontend for running JAX and OpenXLA models on Tenstorrent hardware.

TT-NN™

High-level Python API of pre-optimized neural-network operations.

TT-Lang™

Python DSL for authoring fused custom operations.

TT-MLIR™

MLIR-based backend compiler infrastructure that lowers models onto Tenstorrent hardware.

TT-Metalium™

Low-level C++ SDK for writing custom kernels on Tensix cores.

Cloud-Native Support

Run Tenstorrent accelerators on Kubernetes — install, configure, and operate the device stack.

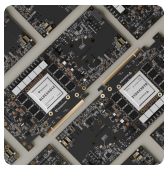
Hardware →

BLACKHOLE



TT-QuietBox® 2 (Blackhole)

Next-generation liquid-cooled Blackhole workstation.



Blackhole® PCIe Cards

Infinitely scalable, high-performance AI processing.



TT-QuietBox® (Blackhole)

Dead-quiet, liquid-cooled desktop AI.



Tenstorrent Galaxy® (Blackhole)

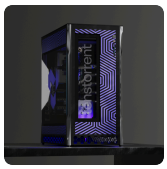
Scalable, enterprise-class AI server.

WORMHOLE



Wormhole™ PCIe Cards

Flexible, scalable processors built with Tensix Cores™.



TT-QuietBox® (Wormhole)

Dead-quiet desktop AI.



TT-LoudBox™ (Wormhole)

High-throughput workstation AI.



Tenstorrent Galaxy® (Wormhole)

Scalable, enterprise-class AI server.

Learn End-to-End through TT-VSCode-Toolkit →

Your First Inference

Set up the toolkit, detect hardware, and run your first model end-to-end.

Serving & APIs

Production inference with tt-inference-server and vLLM, plus image and video generation.

Applications

Real-world apps: coding assistants, AI agents, and video generation on QuietBox 2.

Custom Training

Datasets, fine-tuning, multi-device training, and training from scratch.

Cookbook

Hands-on projects: Game of Life, audio processing, fractals, and particle sims.

Compilers & Frameworks

Image classification with TT-Forge, JAX and PyTorch/XLA, and an intro to TT-Lang.

CS Fundamentals

RISC-V architecture, memory hierarchy, parallelism, synchronization, and complexity.

Advanced

Model bring-up bounties and exploring TT-Metalium at the kernel level.

Deployment

Deploy the toolkit and your work to the cloud with Koyeb.

Tools →

TT-Inference-Server

Deploy and serve models — the canonical source for validated model support.

TT-Studio

Point-and-click web UI for deploying a model — the fastest path for new users.

TT-SMI

Command-line device telemetry and firmware info — ships with the software stack.

TT-Toplike

Real-time terminal monitor: power, temperature, current, and core activity.

TT-NN Visualizer

Inspect model execution: graphs, memory plots, tensor and buffer views.

TT-Topology

Configure Ethernet routing on multi-card QuietBox and LoudBox systems.

TT-VSCode-Toolkit

Guided VS Code learning environment — browse tracks in the Learn section above.

TT-Blacksmith

Optimized ML training recipes for Tenstorrent hardware.

Explore the Ecosystem →

TT-Awesome

A curated directory of 115+ community and official projects, models, kernels, and research across 12 categories.

🪐 Planet Tenstorrent

The latest across the ecosystem — releases, papers, talks, videos, and demos, aggregated in one feed.

Tenstorrent on GitHub

All public source repositories — the software stack, compilers, tools, and demos.

Resources →



Request Support

Submit a support ticket and our team will help.



FAQ

Learn more about our hardware and software.



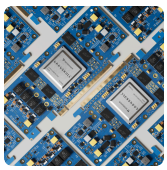
Contact Us

Get in touch with the Tenstorrent team.

Join Our Discord

Join the Tenstorrent developer community.

Legacy



Grayskull™

Tenstorrent's first-generation Tensix processor.



T7000

Earlier-generation Tenstorrent system.



T1000

Earlier-generation Tenstorrent system.